Minder Beta: plan para convertir el prototipo en una aplicación web publicada

Equipo: Luis Izquierdo, Ángel Madrigal y Fabián Sánchez · Octubre de 2026 · 4 semanas, 2 h por semana por persona

Resumen

- **Tiempo real: 24 horas en total** (3 personas × 2 h × 4 semanas), contando instalar, aprender, programar y publicar. Solo alcanza con versiones mínimas de cada módulo.
- **Stack:** HTML + CSS + JavaScript básico (frontend), Java 21 + Javalin con Maven (backend), PostgreSQL en Neon (base de datos) y Render (publicación).
- **Una regla que se dobla:** Render no ejecuta Java de forma nativa, así que se necesita un `Dockerfile` de unas 8 líneas. **No se instala ni se ejecuta Docker**: Render lo lee y construye por ustedes.
- **Alcance elegido:** lo esencial (cuentas, recordatorios, contactos) + directorio de farmacias y pedidos.
- Los datos de planes gratuitos se verificaron en octubre de 2026 y pueden cambiar; revisen las páginas oficiales antes de empezar.

1. Herramientas

Capa	Herramienta	Qué aprender
a) Frontend	HTML + CSS + JavaScript básico (su prototipo)	`fetch()` y manipular la página
b) Backend	Java 21 + Javalin (con Maven)	rutas, JSON, sesión
c) Base de datos	PostgreSQL en Neon	SQL básico y JDBC
d) Publicación	Render, conectado a GitHub	variables de entorno y logs

Git y GitHub no son una capa, pero sin ellos no se puede publicar.

a) Frontend: su prototipo + un poco de JavaScript

- **Por qué es la más fácil:** ya tienen el HTML. El servidor Java entrega esas mismas páginas desde una carpeta `public/`, así que hay un solo servidor y un solo despliegue. Nada de React.
- **Qué aprender (cerca de 1 hora, durante el Módulo 1):** `fetch()` para pedir datos al backend (un patrón que se copia y adapta), crear elementos en la página con JavaScript, y la Consola y la pestaña Red del navegador (tecla F12).
- **Instalación:** ninguna. Usan VS Code y Chrome o Edge.

b) Backend: Java + Javalin

- **Por qué es la más fácil:** Javalin es una librería pequeña, sin anotaciones ni configuración: una ruta es una línea, como `app.get("/api/farmacias", ctx -> ...)`. Arranca desde un `main` normal, igual que sus ejercicios, y no es Spring. Usen la serie 6.x (la documentación muestra la 6.7.0; confirmen la última 6.x en javalin.io).
- **Qué aprender:** rutas GET/POST/PUT/DELETE, leer JSON, la sesión del usuario, y de Maven solo el `pom.xml` y 3 comandos.
- **Instalación en Windows:**

1. **Todos, JDK 21:** descarguen Temurin 21 (.msi) en adoptium.net. En el instalador activen "Set JAVA_HOME" y "Add to PATH". Abran una terminal nueva y escriban `java -version`.
2. **Todos, VS Code:** instalen la extensión "Extension Pack for Java" (Microsoft) y reinicien VS Code.
3. **Todos, Git:** descarguen Git for Windows en git-scm.com con las opciones por defecto y verifiquen con `git --version`. Configuren `git config --global user.name "Su Nombre"` y `git config --global user.email "correo-de-github"`.
4. **Solo Luis, Maven:** en maven.apache.org/download.cgi descarguen el "Binary zip archive" y descomprímanlo en `C:\Tools\`. En "Editar las variables de entorno del sistema → Path → Nuevo" agreguen la carpeta `bin` de Maven. Verifiquen con `mvn -v` en una terminal nueva. Luego Luis ejecuta `mvn wrapper:wrapper`, que agrega `mvnw.cmd` al repositorio. Ángel y Fabián usan `.\mvnw.cmd` y no instalan Maven.

c) Base de datos: PostgreSQL en Neon

- **Por qué es la más fácil:** no se instala nada, la crean desde el navegador, y el SQL que aprendan sirve en cualquier otro lugar.
- **Por qué no las otras:**
  - **SQLite:** el archivo viviría en el disco de Render, que en el plan gratuito es efímero y no admite discos persistentes. Perderían los datos.
  - **Postgres de Render:** el gratuito expira a los 30 días de creado.
  - **Supabase:** pausa los proyectos tras 7 días de inactividad.
  - **Neon:** su plan gratuito es permanente, con 0,5 GB por proyecto y 100 horas de cómputo al mes, sin tarjeta. Se duerme tras 5 minutos sin uso y se reanuda en una fracción de segundo.
- **Qué aprender:** SQL básico (`CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`, `WHERE`, `JOIN`) y, en Java, JDBC con `PreparedStatement`: siempre con `?`, nunca pegando texto del usuario dentro del SQL.
- **Instalación (Ángel, semana 1):**
  1. Entren a neon.com con "Sign up" usando su cuenta de GitHub.
  2. Creen un proyecto llamado `minder` y elijan la región más cercana a Colombia que aparezca.
  3. En el botón "Connect" copien la cadena de conexión.
  4. Abran el "SQL Editor" y creen una tabla de prueba.
  5. Inviten a Luis y Fabián al proyecto o compártanles la cadena por un chat privado.
- **Regla de oro:** la clave nunca va en el código ni en GitHub. Va en variables de entorno (`DB_URL`, `DB_USER`, `DB_PASSWORD`). Neon da la cadena como `postgresql://...`, pero Java necesita `jdbc:postgresql://...` con usuario y clave por separado; esto se resuelve en el Módulo 0.

d) Publicación: Render

- **Por qué es la más fácil:** ofrece 750 horas de instancia al mes y no pide tarjeta. Se vuelve a publicar solo cada vez que hacen `git push`.
- **Alternativas descartadas:** Koyeb pide tarjeta desde febrero de 2026; Railway da una prueba de $5 por 30 días y luego solo $1 de crédito al mes.
- **El costo:** el servicio se duerme tras 15 minutos sin visitas. Con Java, la primera visita después puede tardar cerca de un minuto o más. **Antes de exponer, abran la página 5 minutos antes.**
- **Instalación (Fabián, semana 1, cuando exista el repositorio):**

1. En render.com usen "Get Started" con su cuenta de GitHub.
2. Elijan New → Web Service y seleccionen el repositorio `minder`.
3. En Language elijan Docker y en Instance Type elijan Free.
4. Agreguen las 3 variables de entorno de la base de datos.
5. Presionen Create Web Service y esperen al "Live".
6. Abran la URL `https://…onrender.com`.

Git: el ciclo diario

`git pull` → trabajar → `git add .` → `git commit -m "mensaje"` → `git push`. Luis crea el repositorio privado `minder` en GitHub, invita a los otros dos y todos hacen `git clone`.

2. Cómo organizar el código

Clases, con una carpeta por función. La ficha ya está en clases y la presentación pide mostrar herencia y encapsulamiento; con funciones sueltas eso no se vería.

- `modelo/`: las clases de la ficha (Persona, Usuario, Farmacia…), con atributos privados, getters/setters y las reglas, como la validación del teléfono.
- `repositorio/`: una clase por tabla (`UsuarioRepositorio`…). **Solo estas clases escriben SQL.**
- `App.java`: las rutas. Recibe la petición, llama a las clases y responde JSON.

Los métodos de la ficha se reparten así: los que guardan, editan o eliminan (`registrar()`, `eliminar()`…) los hace el repositorio; las reglas (`verificarHorario()`, `posponer()`, `cancelar()`) viven en la clase. Es una adaptación de la ficha y deben poder explicarla en la exposición.

3. Base de datos

```
usuarios(id, nombre, documento*, edad, telefono, correo*, contrasena_hash, latitud, longitud, esta_conectado, rol, activo)
contactos_emergencia(id, usuario_id→usuarios, nombre, documento?, telefono, parentesco?)
recordatorios(id, usuario_id→usuarios, titulo, descripcion, fecha_hora, cumplido, creado_en)
farmacias(id, nombre, tipo, direccion, telefono, latitud, longitud, hora_apertura, hora_cierre, abierto_24h,
acepta_domicilios, usuario_id→usuarios?)
pedidos(id, usuario_id→usuarios, farmacia_id→farmacias, descripcion, direccion_entrega, estado, tiempo_entrega_min?,
creado_en, actualizado_en)
```

`*` = único, `?` = opcional, `→` = apunta a otra tabla.

Decisiones, en lenguaje sencillo:

- **Persona no tiene tabla.** Es una clase "molde" que solo existe en Java. Usuario y ContactoEmergencia heredan de ella en Java, pero en la base de datos cada una tiene su propia tabla con sus columnas. Es lo más simple y mantiene la herencia en el código.
- `id` **automático:** un número que Postgres asigna solo y sirve para que otras tablas apunten a esa fila.
- `contrasena_hash`**:** nunca se guarda la contraseña real, sino un "resumen" que no se puede revertir (con BCrypt). En Java la clase recibe la contraseña y la convierte antes de guardarla.
- **Teléfono con doble candado:** el setter de Java lo valida y la base de datos también (`CHECK`: exactamente 10 dígitos).
- `rol` (`cliente`, `administrador` o `farmacia`) permite un solo login para los tres tipos de agente. Los contactos de emergencia no tienen cuenta.
- `activo`**:** `activarCuenta()` y `suspenderCuenta()` se reducen a cambiar este campo.

- `esta_conectado`**:** se actualiza al iniciar y cerrar sesión. No es 100 % fiable (si cierran el navegador sigue en verdadero), pero coincide con su ficha.
- `farmacias.tipo`**:** `drogueria` u `hospital`, así una sola tabla sirve para el directorio. `acepta_domicilios`**:** solo las droguerías reciben pedidos.
- **Horario:** una sola hora de apertura y una de cierre, más `abierto_24h`. "Abierta ahora" **no se guarda**: se calcula cada vez con `verificarHorario()`.
- `farmacias.usuario_id`**:** la cuenta con rol `farmacia` que administra ese establecimiento.
- `pedidos.estado`**:** `enviado`, `confirmado`, `rechazado`, `entregado` o `cancelado`. Se juntaron `crear()` y `enviarSolicitud()`: el pedido nace "enviado".
- **Relaciones "uno a muchos":** un usuario tiene muchos recordatorios, muchos contactos y muchos pedidos; una farmacia tiene muchos pedidos. Cada fila "hija" guarda el `usuario_id` o `farmacia_id` de su dueño. Es la versión en tablas de sus relaciones "TIENE UN".
- `latitud` **y** `longitud` existen para que coincida con la ficha, pero "las más cercanas" queda recortado.

Usen datos inventados en todas las pruebas. La app maneja ubicación y salud, y no deben subir datos reales de nadie.

4. Módulos y orden

Todos siguen el mismo patrón de 5 pasos: **tabla → clase → repositorio → ruta → pantalla**.

#	Módulo	Por qué en este lugar
0	Hola publicado	Sin esto nada se ve ni se prueba.
1	Directorio de farmacias	Solo lectura y sin login. Prueban toda la cadena (pantalla, lógica, datos) con lo más fácil.
2	Cuentas (herencia Persona → Usuario)	Lo más difícil en conceptos (sesión y hash), pero los módulos 3 a 5 necesitan saber "quién es".
3	Recordatorios	Repite el patrón del Módulo 1 y le suma el dueño.
4	Contactos (Persona → ContactoEmergencia)	Mismo patrón otra vez, así que sale más rápido.
5	Pedidos	Une usuario y farmacia, tiene estados y dos tipos de usuario. Es lo más complejo.

5. Ruta semana a semana

Semana	Trabajo	Horas
1	Módulo 0	6
2	Módulo 1 + primera parte del Módulo 2	6
3	Fin del Módulo 2 + Módulo 3 + inicio del Módulo 4	≈6
4	Fin del Módulo 4 + Módulo 5 + cierre	≈6

Dentro de cada tarea incluyan 10 minutos de coordinación por chat o llamada. Al empezar cada módulo, escriban en el README el "contrato": las rutas y los nombres exactos de los campos JSON. Los nombres que no coinciden entre JavaScript y Java causan la mayoría de los errores entre compañeros. Cada módulo se publica en cuanto termina.

Módulo 0: Hola publicado (Semana 1)

Tarea	Responsable	Tiempo
Instalar JDK, extensión Java y Git; configurar Git; aceptar la invitación al repo	Todos	45 min c/u
Instalar Maven, crear el repo y el proyecto Javalin que sirva el prototipo (`public/index.html`), generar wrapper y `.gitignore`, subir	Luis	1 h 15
Crear el proyecto en Neon y una tabla de prueba; hacer la ruta `/db` que lea una fila (credenciales en variables de entorno)	Ángel	1 h 15
Crear el servicio en Render con el Dockerfile y las variables; desplegar	Fabián	1 h 15

Luis sube primero. Mientras tanto, Ángel y Fabián abren sus cuentas y esperan su aviso.

- **Al terminar debe funcionar:** la URL pública muestra su prototipo, `/db` muestra un dato leído de Neon y los tres corren el proyecto en `localhost:7070`.
- **Cómo comprobarlo:** abran la URL en un celular con datos móviles. Cambien una palabra, hagan `git push` y verifiquen que aparece en pocos minutos.
- **Error más común:** Java escucha en un puerto fijo y Render dice que no encuentra el servicio. Se evita leyendo el puerto de la variable `PORT` (`System.getenv("PORT")`, con 7070 por defecto en local).

Módulo 1: Directorio de farmacias (Semana 2)

Tarea	Responsable	Tiempo
Tabla `farmacias` y 8 filas de ejemplo; guardar el script en `sql/01_farmacias.sql`	Luis	1 h
Clase `Farmacia` (atributos privados, `verificarHorario()`), `FarmaciaRepositorio.listar()` y ruta `GET /api/farmacias`	Ángel	1 h
Mover el CSS del prototipo a `estilos.css`; `farmacias.html` con `fetch` que dibuja tarjetas con nombre, teléfono (enlace `tel:`), "Abierta/Cerrada" y botón "Ver en Google Maps"	Fabián	1 h

- **Al terminar debe funcionar:** la lista aparece en la URL pública con el estado correcto de cada farmacia.
- **Cómo comprobarlo:** en Neon cambien `hora_cierre` de una farmacia a una hora ya pasada, recarguen y debe pasar a "Cerrada".
- **Error más común:** usar la hora del servidor, que está en UTC, y mostrar "Abierta" a deshoras. Se evita usando `ZoneId.of("America/Bogota")` al comparar.

Módulo 2: Cuentas (Semanas 2 y 3)

Tarea	Responsable	Tiempo
Tabla `usuarios` (con `CHECK` del teléfono) y `UsuarioRepositorio` (guardar, buscar por correo)	Luis	1 h
`Persona` (abstracta, setter del teléfono) y `Usuario extends Persona`	Ángel	1 h
`registro.html` y `login.html`	Fabián	1 h
Rutas `/api/registro`, `/api/login`, `/api/logout`, `/api/yo`, con sesión y BCrypt	Ángel	1 h
`sesion.js` compartido: si no hay sesión, redirige al login; muestra el nombre y "Cerrar sesión"	Luis	1 h

- **Al terminar debe funcionar:** registrarse, cerrar sesión e iniciar sesión; un teléfono de 9 dígitos se rechaza con mensaje; las páginas personales mandan al login si no hay sesión.
- **Cómo comprobarlo:** en Neon la contraseña debe verse como texto ilegible. Registren una cuenta y verifiquen cada caso.
- **Error más común:** guardar la contraseña tal cual. Se evita llamando siempre a BCrypt antes de guardar.
- **No es un error:** cuando Render se duerme, la sesión se pierde y hay que iniciar sesión otra vez.

Módulo 3: Recordatorios (Semana 3)

Tarea	Responsable	Tiempo
Tabla `recordatorios`, clase `Recordatorio` (`marcarCumplido()`, `posponer()`) y repositorio	Fabián	1 h
Rutas: listar, crear, editar, eliminar, cumplir y posponer, siempre del usuario de la sesión	Luis	1 h
`recordatorios.html`: formulario con `datetime-local`, lista, botones, y un aviso en pantalla cuando llega la hora (revisión cada 30 s con la página abierta)	Ángel	1 h

- **Al terminar debe funcionar:** crear, ver, editar, borrar, marcar cumplido y posponer 10 minutos. Cada usuario ve solo los suyos.
- **Cómo comprobarlo:** inicien sesión con dos cuentas (una en ventana normal y otra en incógnito) y verifiquen que no ven los recordatorios del otro.
- **Error más común:** el navegador manda la hora de Colombia y el servidor, en UTC, la interpreta mal; el aviso llega 5 horas tarde. Se evita enviando la fecha en formato ISO con `new Date(valor).toISOString()` y guardando la columna como `TIMESTAMPTZ`.

Módulo 4: Contactos de emergencia (Semanas 3 y 4)

Tarea	Responsable	Tiempo
Tabla `contactos_emergencia`, clase `ContactoEmergencia extends Persona` y repositorio	Luis	45 min
Rutas de registrar, editar, eliminar y listar	Ángel	45 min
`contactos.html` con un enlace "Llamar" por contacto, más el botón del 123 del prototipo (`tel:123` con confirmación)	Fabián	1 h

- **Al terminar debe funcionar:** CRUD de contactos por usuario y botón 123.
- **Cómo comprobarlo:** prueben el botón y los enlaces `tel:` desde un celular, porque en el computador pueden no hacer nada.
- **Error más común:** validar el teléfono solo en JavaScript. Se evita usando el setter de `Persona`, que ya valida; así Usuario y ContactoEmergencia comparten la misma regla gracias a la herencia.

Módulo 5: Pedidos de domicilio (Semana 4)

Tarea	Responsable	Tiempo
Tabla `pedidos`, clase `Pedido` (guarda un `Usuario` y una `Farmacia`; `cancelar()` solo si está "enviado") y repositorio	Luis	55 min

Tarea	Responsable	Tiempo
Rutas: crear pedido, mis pedidos, cancelar; y para rol `farmacia`: sus pedidos, confirmar con tiempo de entrega, rechazar y marcar entregado	Ángel	55 min
Pantallas: botón "Pedir domicilio" en cada droguería, "Mis pedidos" y un panel mínimo de farmacia	Fabián	1 h 40

- **Al terminar debe funcionar:** un cliente crea un pedido, la farmacia lo confirma con un tiempo de entrega y el cliente ve el estado actualizado.
- **Cómo comprobarlo:** creen a mano la cuenta de farmacia en Neon (`UPDATE usuarios SET rol='farmacia'` y `UPDATE farmacias SET usuario_id=...`). Con dos navegadores (normal e incógnito), el cliente pide, la farmacia confirma 30 min y el cliente recarga y ve "Confirmado, 30 min".
- **Error más común:** no revisar quién puede hacer qué; cualquiera podría cancelar el pedido de otro cambiando el id. Se evita comprobando en cada ruta que el pedido pertenezca al usuario de la sesión y que el cambio de estado sea válido.

Cierre (Semana 4, 20 min por persona)

Prueben todo de principio a fin en la URL pública con cuentas nuevas y escriban el README con qué hace, la URL y los integrantes. Ensayen la explicación de la herencia (`Persona → Usuario` y `Persona → ContactoEmergencia`): todos deben poder explicarla porque todos la implementaron.

6. Qué recortar

Ya recortado desde el inicio, porque no cabe en 24 h:

- **Asistente de voz (Beta):** nada esencial depende de él. Déjenlo como trabajo futuro.
- **Modo sin conexión (2.6):** necesita almacenamiento local, un service worker y sincronización. Es un proyecto aparte; documéntenlo como trabajo futuro.
- **Mapa interactivo con "las más cercanas":** se reemplaza por una lista con "abierta ahora" y un enlace a Google Maps.
- **Avisos reales a contactos (SMS o notificación) y alarmas con la app cerrada.**
- **Panel de administrador y sugerencias o reportes:** el administrador trabaja con el SQL Editor de Neon (por ejemplo `UPDATE usuarios SET activo=false`).

Si se atrasan, recorten en este orden:

1. El panel de farmacia del Módulo 5 (la farmacia cambia el estado con un `UPDATE` en Neon durante la demo).
2. "Posponer" y el aviso en pantalla del Módulo 3.
3. Editar contactos (dejen solo crear y borrar).
4. El Módulo 5 completo.

Es mejor tener 5 módulos terminados y publicados que 6 a medias.

7. Siguiente paso

Instalen todo en la semana 1. Cuando lo tengan listo, pidan el **Módulo 0** y recibirán el `pom.xml`, `App.java` y el `Dockerfile`, con los comandos exactos y explicados línea por línea. El código se pide módulo por módulo.